

PROYECTO FINAL

Curso: Sistemas Operativos II

Institución: Universidad Mariano Gálvez de Guatemala

Facultad: Ingeniería en Sistemas de Información

Plataforma Cloud: Amazon Web Services (AWS)

Fecha: May 2026

1. Introducción

El presente documento describe el diseño, implementación y despliegue de una infraestructura moderna basada en prácticas DevOps. El proyecto integra tecnologías reales utilizadas en la industria: contenedores Docker, orquestación con Docker Swarm, pipeline CI/CD automatizado con GitHub Actions, y despliegue en Amazon Web Services (AWS).

1.1 Objetivos

- Diseñar e implementar una arquitectura cloud moderna en AWS
- Containerizar una aplicación web usando Docker y Docker Compose
- Implementar orquestación de contenedores con Docker Swarm
- Construir un pipeline CI/CD completo con GitHub Actions
- Garantizar disponibilidad y seguridad básica de la infraestructura
- Documentar la arquitectura y evidenciar el funcionamiento del sistema

1.2 Descripción de la empresa ficticia

TechFlow Guatemala S.A. es una empresa ficticia de desarrollo de software que necesita migrar sus aplicaciones legacy a la nube y modernizar su proceso de desarrollo. El equipo de DevOps tiene como misión diseñar la nueva infraestructura cloud, automatizar los despliegues y garantizar la disponibilidad del sistema en producción.

2. Diseño Arquitectónico

2.1 Diagrama de Arquitectura Cloud

Componente	Tecnología	Función
DNS / Internet	IP Pública AWS	Punto de entrada del tráfico
Load Balancer / Proxy	Nginx 1.25 (container)	Distribuye tráfico a la app
Aplicación Web	Python Flask (container)	Lógica de negocio y API REST
Base de Datos	PostgreSQL 15 (container)	Persistencia de datos
Red Virtual	VPC + Security Group AWS	Aislamiento y seguridad
Instancia Cloud	EC2 t2.micro Ubuntu 22.04	Servidor de producción
Repositorio	GitHub	Control de versiones y CI/CD
Registry	Docker Hub	Almacenamiento de imágenes
Monitoreo	Docker Health Checks	Estado de los contenedores

Tabla 1: Componentes de la arquitectura cloud

2.2 Flujo de arquitectura

El flujo de tráfico en producción sigue el siguiente camino: el usuario realiza una petición HTTP al dominio o IP pública del servidor EC2. El tráfico entra al puerto 80, donde Nginx actúa como reverse proxy y lo reenvía al contenedor Flask en el puerto 5000. Flask procesa la solicitud, consulta PostgreSQL si es necesario, y retorna la respuesta. Todo esto ocurre dentro de una red Docker overlay aislada.

Paso	Origen	Destino	Puerto
1	Usuario / Internet	EC2 Nginx	80 (HTTP)
2	Nginx (Proxy)	Flask App	5000 (interno)
3	Flask App	PostgreSQL	5432 (interno)
4	Flask App	Nginx / Usuario	Respuesta HTTP

Tabla 2: Flujo de tráfico en producción

3. Configuración Cloud – Amazon Web Services

3.1 Servicios AWS utilizados

Servicio	Descripción
EC2 t2.micro	Instancia de cómputo en Free Tier – Ubuntu 22.04 LTS
VPC	Red virtual privada para aislar los recursos cloud
Security Group	Firewall virtual con reglas de entrada/salida
Elastic IP	IP pública estática asignada a la instancia
Key Pair (.pem)	Par de claves RSA para acceso SSH seguro

3.2 Configuración de red

La instancia EC2 se despliega dentro de una VPC con una subred pública. El Security Group permite tráfico entrante únicamente en los puertos necesarios:

Puerto	Protocolo	Fuente	Propósito
22	TCP	IP del administrador	Acceso SSH seguro
80	TCP	0.0.0.0/0	Tráfico HTTP público
443	TCP	0.0.0.0/0	HTTPS (opcional, puntos extra)

3.3 Acceso SSH

El acceso a la instancia se realiza mediante par de claves RSA (archivo .pem):

```
# Asegurar permisos del archivo de clave
chmod 400 mi-clave-devops.pem

# Conectar a la instancia EC2
ssh -i mi-clave-devops.pem ubuntu@TU_IP_PUBLICA_EC2
```

4. Contenedores Docker

4.1 Dockerfile – Aplicación Flask

Se utilizó un Dockerfile multi-stage para optimizar el tamaño de la imagen final. La primera etapa (builder) instala las dependencias; la segunda etapa (runtime) copia solo lo necesario, sin herramientas de build. Se ejecuta como usuario no-root por seguridad.

```
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

FROM python:3.11-slim
RUN addgroup --system appgroup && adduser --system --ingroup appgroup appuser
WORKDIR /app
COPY --from=builder /usr/local/lib/python3.11/site-packages .
COPY app.py .
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s CMD python -c "import urllib.request; urllib.request.urlopen(.
..)"
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "app:app"]
```

4.2 Docker Compose

Docker Compose define los tres servicios del stack: la aplicación Flask, Nginx como reverse proxy y PostgreSQL como base de datos. Los servicios se comunican a través de una red bridge interna.

```
services:
  app:
    build: ./app
    networks: [app_network]
    healthcheck: ...

  nginx:
    image: nginx:1.25-alpine
    ports: ["80:80"]
    depends_on: {app: {condition: service_healthy}}

  db:
    image: postgres:15-alpine
    environment: [POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD]
    volumes: [db_data:/var/lib/postgresql/data]

networks:
  app_network:
    driver: bridge
```

```
ipam:
  config: [{subnet: 172.20.0.0/24}]
```

4.3 Docker Swarm – Orquestación

Para orquestación de contenedores se implementa Docker Swarm, que permite ejecutar múltiples réplicas de la aplicación con balanceo de carga automático, actualizaciones sin downtime (rolling updates) y reinicio automático ante fallos.

```
# Inicializar el swarm
docker swarm init

# Desplegar el stack
docker stack deploy -c swarm-stack.yml devops

# Ver servicios corriendo
docker service ls

# Escalar la aplicación a 3 réplicas
docker service scale devops_app=3
```

Característica	Valor configurado
Réplicas de la app	2 (escalable)
Estrategia de update	Rolling (1 a la vez)
Política de reinicio	on-failure (máx. 3 intentos)
Placement de nginx	Solo en nodo manager
Secrets	Contraseña DB como Docker secret

5. Pipeline CI/CD – GitHub Actions

El pipeline de integración y entrega continua se implementa con GitHub Actions. Se activa automáticamente con cada push a la rama main. El pipeline tiene dos jobs: CI (integración continua) y CD (entrega continua).

5.1 Job CI – Integración Continua

El job de CI ejecuta las siguientes etapas en secuencia:

Etapas	Descripción
Checkout	Descarga el código fuente del repositorio
Setup Python	Configura Python 3.11 en el runner de GitHub
Install dependencies	Instala Flask, Gunicorn y pytest
Run tests	Ejecuta tests básicos contra los endpoints de la app
Build Docker image	Construye la imagen Docker con el tag de versión
Test container	Levanta el contenedor y verifica el health check

5.2 Job CD – Entrega Continua

El job de CD solo se ejecuta en pushes a main (no en pull requests). Requiere que el job CI complete exitosamente:

Etapas	Descripción
Login DockerHub	Autenticación con credenciales almacenadas como Secrets
Push imagen	Publica la imagen en DockerHub con tag de versión y latest
SSH a EC2	Conexión segura a la instancia de producción
docker compose down/up	Actualiza los servicios con la nueva imagen
Health check	Verifica que el deploy fue exitoso via curl

5.3 Secrets de GitHub Actions

Las credenciales se almacenan de forma segura en GitHub Secrets:

Secret	Descripción
DOCKERHUB_USERNAME	Nombre de usuario de DockerHub

DOCKERHUB_TOKEN	Token de acceso a DockerHub (no password)
EC2_HOST	IP pública de la instancia EC2
EC2_SSH_KEY	Contenido del archivo .pem de AWS

5.4 Fragmento del pipeline YAML

```
name: CI/CD Pipeline
on:
  push:
    branches: [main]

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker image
        run: docker build -t $IMAGE:$VERSION ./app
      - name: Test container
        run: |
          docker run -d -p 5000:5000 $IMAGE:$VERSION
          curl -f http://localhost:5000/health

  cd:
    needs: ci
    steps:
      - uses: docker/login-action@v3
      - uses: docker/build-push-action@v5
        with: {push: true}
      - name: Deploy to EC2
        uses: appleboy/ssh-action@v1.0.0
        with:
          script: docker compose down && docker compose up -d
```

6. Monitoreo y Seguridad

6.1 Monitoreo implementado

El monitoreo se implementa a través de Docker Health Checks nativos, que verifican el estado de cada contenedor de forma periódica. Los health checks están configurados en el Dockerfile y en Docker Compose.

```
# Verificar estado de los contenedores
docker compose ps

# Ver logs en tiempo real
docker compose logs -f app

# Endpoint de health check de la aplicacion
curl http://TU_IP/health
# Respuesta: {"status": "healthy", "hostname": "...", "timestamp": "..."}
```

Herramienta	Tipo	Descripción
Docker Health Check	Nativo	Verifica /health cada 30 segundos
docker compose ps	Manual	Estado de todos los contenedores
docker compose logs	Manual	Logs de aplicación en tiempo real
Nginx stub_status	Nativo	Métricas de conexiones Nginx

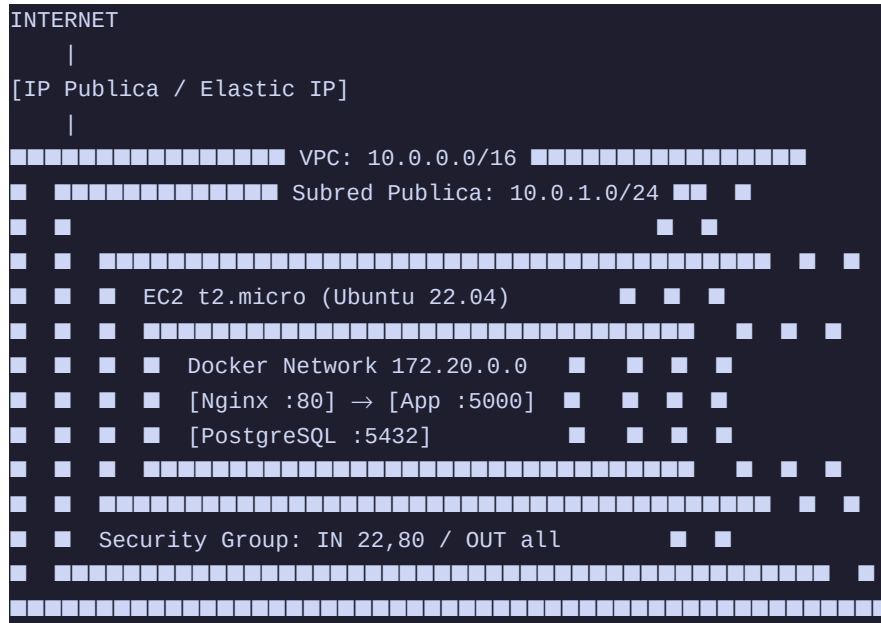
6.2 Seguridad implementada

- Usuario no-root dentro de los contenedores Docker
- Security Group con puertos mínimos necesarios (22, 80)
- Acceso SSH con par de claves RSA (sin password)
- Credenciales almacenadas en GitHub Secrets (nunca en código)
- Headers de seguridad HTTP configurados en Nginx (X-Frame-Options, X-XSS-Protection)
- Variables de entorno para passwords (nunca hardcoded)
- Docker secrets para credenciales de base de datos en Swarm
- Archivo .gitignore que excluye .env y archivos .pem del repositorio

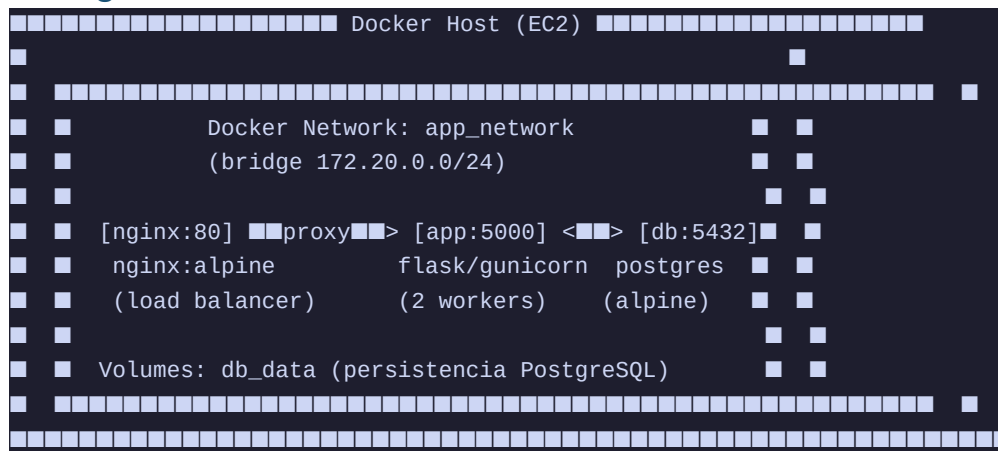
7. Diagramas de Arquitectura

7.1 Diagrama de Red y VPC

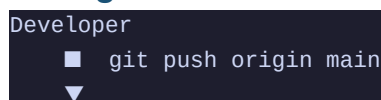
La siguiente representación muestra la segmentación de red en AWS:



7.2 Diagrama de Contenedores



7.3 Diagrama CI/CD



[GitHub Repository]

- webhook trigger


```
[GitHub Actions Runner]
```


■■■ JOB: CI ■■■

- 1. Checkout code
- 2. pip install requirements
- 3. python tests (health check, index)
- 4. docker build -t imagen:version
- 5. docker run + curl /health
- 6. [PASS / FAIL]


```
■■■ JOB: CD (si CI pasa) ■■■
```

1. `docker login DockerHub`
2. `docker push imagen:version + latest`
3. SSH → EC2
4. `docker compose down`
5. `docker compose up -d`
6. `curl /health` → DEPLOY OK

8. Evidencias del Sistema

Las siguientes secciones describen las evidencias recopiladas durante el despliegue. Las capturas de pantalla se obtuvieron durante la ejecución real del pipeline y del sistema en producción.

Pipeline CI/CD ejecutado

Captura del workflow en GitHub Actions mostrando todos los steps completados (check verde)

[Captura de pantalla – Pipeline CI/CD ejecutado]

Imagen publicada en DockerHub

Captura del repositorio en DockerHub con la imagen y el tag de versión

[Captura de pantalla – Imagen publicada en DockerHub]

Instancia EC2 corriendo

Captura de la consola AWS mostrando la instancia en estado Running con IP pública

[Captura de pantalla – Instancia EC2 corriendo]

Contenedores activos

Salida de docker compose ps mostrando los 3 contenedores en estado healthy

[Captura de pantalla – Contenedores activos]

Aplicación accesible

Captura del navegador mostrando la página web en `http://IP_PUBLICA`

[Captura de pantalla – Aplicación accesible]

Health check funcionando

Respuesta JSON del endpoint `/health` con status: healthy

[Captura de pantalla – Health check funcionando]

Logs del pipeline

Logs de GitHub Actions mostrando el SSH deploy y el curl de verificación

[Captura de pantalla – Logs del pipeline]

9. Conclusiones

9.1 Retos encontrados

- Configurar correctamente el Security Group de AWS para permitir únicamente el tráfico necesario
- Establecer la comunicación entre contenedores en la red Docker interna
- Configurar los GitHub Secrets y el acceso SSH desde el runner de GitHub Actions
- Implementar health checks que funcionen correctamente antes de que Nginx dependa del servicio
- Gestionar las variables de entorno de forma segura sin exponerlas en el repositorio

9.2 Aprendizajes

- Docker multi-stage builds reducen significativamente el tamaño de la imagen de producción
- Los health checks nativos de Docker son suficientes para garantizar orden de inicio de servicios
- GitHub Actions elimina la necesidad de un servidor Jenkins dedicado para proyectos pequeños
- Docker Swarm ofrece orquestación básica sin la complejidad operacional de Kubernetes
- La separación de responsabilidades (Nginx, App, DB) en contenedores distintos facilita el mantenimiento
- Almacenar secrets en GitHub y nunca en el código es una práctica fundamental de seguridad DevOps

9.3 Resultado final

Se logró implementar una infraestructura DevOps funcional y completa que cumple con todos los requisitos del proyecto: aplicación containerizada con Docker, orquestación con Docker Swarm, pipeline CI/CD automatizado con GitHub Actions, y despliegue en AWS EC2. El sistema es capaz de actualizar automáticamente la aplicación en producción con cada commit a la rama main, verificando su correcto funcionamiento antes de completar el despliegue.

10. Criterios de Evaluación

Criterio	Descripción	Puntos
Diseño arquitectónico	Diagrama completo con todos los componentes requeridos	
Infraestructura cloud	EC2, VPC, Security Group, IP pública funcional	1
Docker y contenedores	Dockerfile multi-stage, Docker Compose con 3 servicios	
Docker Swarm	swarm-stack.yml con réplicas, rolling update, secrets	
Pipeline CI/CD	GitHub Actions con CI + CD completo y funcional	
Seguridad y redes	Security Group, SSH keys, headers HTTP, secrets management	
Monitoreo	Health checks, logging, endpoint /health	1
Documentación técnica	Este documento con todos los componentes requeridos	
Presentación final	Demo en vivo del pipeline, contenedores e infraestructura	
TOTAL		10